

Lambda Expressions

Java

↳ OO language

Everything in Java
Revolves around Object.

Behaviour/
logic

class X {
 m1();
}

Java → Objects ✓

↳ Behaviour

X x = new X();

↳ Functional Interfaces

↓
Lambda Expressions

Behaviour / logic

↳ logic

Method

class X {

 m1() {

 doSomething();

 }

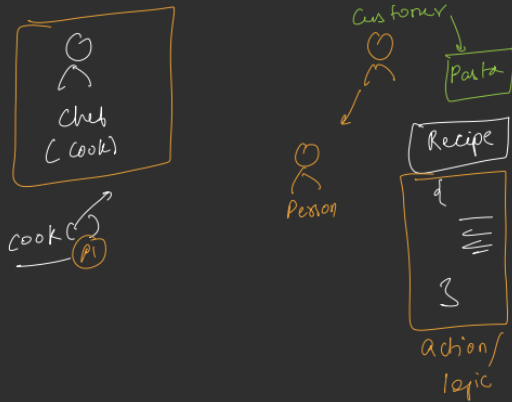
m1()

x1 (Student)

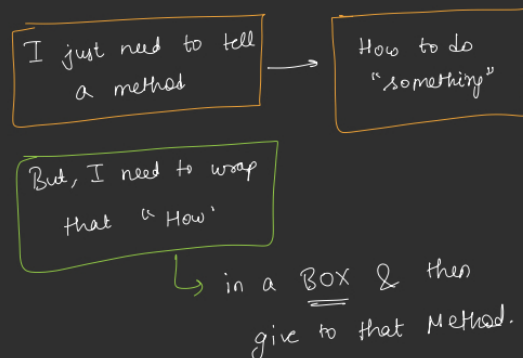
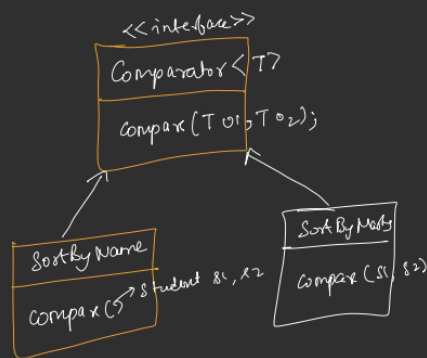
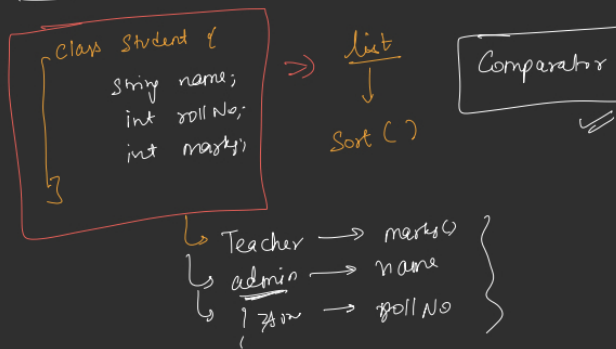
Variable

Primitive

Non-primitive



Why do we need Behaviour Parsing?



Functional Interfaces

↳ only one abstract method.

→ static Method ✓

→ default Method ✓

interface Comparable^T {
 int compareTo(T o);
}

interface Comparator^T {
 int compare(T o1, T o2);
}

Comparator

Class SortByMarks implements Comparator {

 int compare(s1, s2) {
 return s1.marks - s2.marks;
 }
}

Sort(list, —);

Lambda Expression

↳ (Parameters) → expression

↳ Map to / Gives to

Class SortByMarks implements Comparator {
 int compare(s1, s2) {
 return s1.marks - s2.marks;
 }
}

(s1, s2) → s1.marks - s2.marks;

Lambda Expressions

① Multiple parameters

$(a, b) \rightarrow a + b$

② Single parameter.

$x \rightarrow x * x$

③ No parameter.

$() \rightarrow \text{System.out.println("Hello");}$

④ Multi-line

$(a, b) \rightarrow \{$
 $\text{int sum} = a + b;$
 $\text{return sum};$
 $\}$

$(s1, s2) \rightarrow s1.\text{marks} - s2.\text{marks}$

↳ from the current context -

⇒ Target Typing. ✓

Interface I {

f1();

f2();

Lambda $\leftrightarrow$ Functional Interface